

Звіт з лабораторної роботи №4 дисципліни «Основи програмної інженерії»

Тема: Проведення Code Review та рефакторинг програмного забезпечення

Виконавець: студент групи ПЗП-25-3, Федоров К.О.

1. Мета роботи Ознайомлення з процесом колективної розробки, проведенням взаємного Code Review на платформі GitHub та практичне застосування технік рефакторингу для покращення якості програмного забезпечення.

2. Етап 1: Взаємний Code Review (аналіз коду напарника)

У рамках першого етапу було створено Pull Request на GitHub-репозиторії напарника та проведено детальне рев'ю файлу database.py. Під час аналізу виявлено 5 проблем різних категорій; на кожну залишено коментар із рекомендацією щодо виправлення.

- Зауваження №1 — Resource Leak (Витік ресурсів)

- **Проблема:** Відсутність механізму закриття з'єднань. Клас створює нові підключення до БД для потоків, але ніде їх не закриває. Це може призвести до блокувань бази або вичерпання ліміту файлових дескрипторів.
- **Рекомендація:** Додати метод close() для явного закриття з'єднань потоку, або реалізувати магічні методи __enter__ та __exit__ для використання класу як менеджера контексту (with LocalDatabase() as db:).

- Зауваження №2 — Error Handling / Transaction Management (Керування транзакціями)

- **Проблема:** Ручне керування транзакціями без обробки помилок. Використовується явний виклик self.conn.commit(). Якщо під час INSERT станеться помилка (наприклад, порушення UNIQUE constraint), програма аварійно завершиться, а транзакція залишиться відкритою через відсутність rollback().
- **Рекомендація:** Використовувати вбудований менеджер контексту з'єднання: with self.conn: self.conn.execute(...). Це автоматично здійснить commit при успіху або rollback при помилці.

- **Зауваження №3 — Hardcoded Configuration (Жорстко заcodedані налаштування)**
 - **Проблема:** Жорстко заcodedане значення (Magic String). Шлях до бази даних за замовчуванням "garden.db" вписаний безпосередньо в сигнатуру методу `__init__`.
 - **Рекомендація:** Винести назву бази даних у константу рівня модуля (наприклад, `DEFAULT_DB_PATH = "garden.db"`) або використовувати змінні середовища за допомогою `os.getenv`.
- **Зауваження №4 — Performance (Продуктивність)**
 - **Проблема:** Відсутність індексів для зовнішніх ключів. У схемі створюються зв'язки FOREIGN KEY (наприклад, `user_id` у таблиці `contacts`), але для них не створено індексів. Це може суттєво уповільнити виконання операцій JOIN та каскадного видалення при зростанні обсягу даних.
 - **Рекомендація:** Додати створення індексів до методу `_init_db`. Наприклад: `CREATE INDEX IF NOT EXISTS idx_contacts_user_id ON contacts(user_id);`.
- **Зауваження №5 — Maintainability / Incomplete Documentation (Підтримуваність)**
 - **Проблема:** Відсутність документації до публічних методів. Клас та властивість `conn` мають `docstrings`, але робочі методи (`insert_user`, `get_user_by_email` тощо) не задокументовані, що ускладнює розуміння коду іншими розробниками.
 - **Рекомендація:** Додати `docstrings` згідно зі стандартом PEP 257 до кожного публічного методу з описом аргументів та типу даних, що повертається.

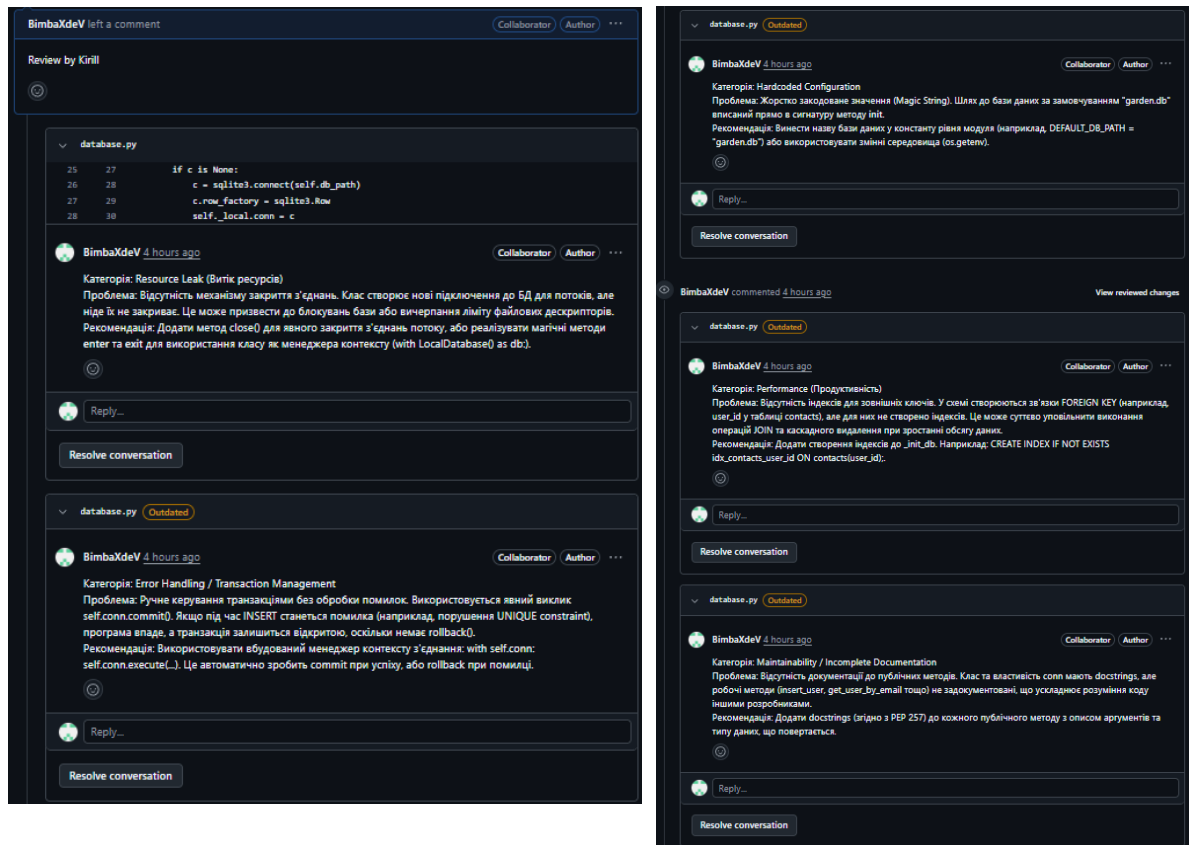


Рис 1-2. Сторінка Pull Request з 5 коментарями на GitHub

3. Етап 2: Рефакторинг власного коду

На основі рев'ю, отриманого від напарника, до файлів `models.py` та `notifications.py` виконано п'ять операцій рефакторингу для усунення архітектурних недоліків.

- **Операція №1: Усунення мертвого коду та порушень форматування**
 - **Категорія:** Dead Code / PEP 8
 - **БУЛО:** Коментар `#Contact` розміщувався без пробілу після решітки, що порушувало стандарт PEP 8 (помилка E265). Крім того, він дублював назву класу, що розміщувався нижче, будучи надлишковим.
 - **СТАЛО:** Зайвий коментар повністю видалено.
 - **ЧОМУ (Обґрунтування):** Очищення коду від інформаційного шуму. Клас вже має змістовний docstring, якого достатньо для документування призначення об'єкта.
- **Операція №2: Заміна неінформативних коментарів**
 - **Категорія:** Погане іменування (Poor Naming)

- **БУЛО:** Використовувався коментар # OPT-4, який нічого не пояснював розробнику без доступу до зовнішньої документації.
- **СТАЛО:** Замінено на семантично зрозумілий коментар: # Перевірка що обидва поля заповнені перед авторизацією.
- **ЧОМУ (Обґрунтування):** Код став самодокументованим, що значно пришвидшує його розуміння та підтримку новими розробниками.
- **Операція №3: Перейменування методів**
 - **Категорія:** Погане іменування (Poor Naming)
 - **БУЛО:** Метод `verify_account` фактично не здійснював повноцінної верифікації (не звертався до БД), а лише перевіряв наявність заповнених полів.
 - **СТАЛО:** Метод перейменовано на `is_credentials_present()` (або `has_required_fields()`).
 - **ЧОМУ (Обґрунтування):** Назва методу тепер точно відповідає його фактичній поведінці, усуваючи неоднозначність в архітектурі системи.
- **Операція №4: Видалення невикористаних параметрів**
 - **Категорія:** Dead Code (SonarQube S1172)
 - **БУЛО:** У сигнатурі методу `check_if_reminder_needed` був оголошений параметр `contact_id`, який не використовувався у його тілі.
 - **СТАЛО:** Параметр `contact_id` вилучено із сигнатури.
 - **ЧОМУ (Обґрунтування):** Видалення "мертвих" параметрів спрощує інтерфейс методу, зменшує когнітивне навантаження та вирішує зауваження статичного аналізатора.
- **Операція №5: Декомпозиція та дотримання принципів SOLID**
 - **Категорія:** Порушення SRP (Single Responsibility Principle)
 - **БУЛО:** Метод `schedule_push` виконував одразу дві різні задачі: перевіряв часові ліміти відправки (rate limit) та безпосередньо формував повідомлення.

- **СТАЛО:** Логіку розділено на два окремих методи: `_is_rate_limited()` для внутрішньої перевірки лімітів та публічний `send_push()` для відправки повідомлень.
- **ЧОМУ (Обґрунтування):** Розподіл відповідальності дозволяє ізольовано покривати логіку лімітів модульними тестами та покращує загальну архітектурну гнучкість модуля.

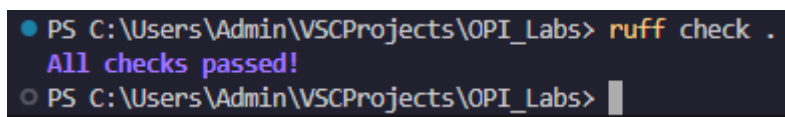
4. Етап 3: Верифікація результатів

Після завершення рефакторингу проведено комплексну перевірку для підтвердження того, що функціональність системи не порушена, а метрики якості коду покращено.

- **4.1. Статичний аналіз (SonarLint):** За допомогою розширення SonarLint проаналізовано оновлену кодову базу. Усі критичні зауваження (Code Smells), проблеми безпеки (Security Hotspots) та попередження про можливий витік ресурсів успішно усунуто.

Результати проведеної роботи з рефакторингу згідно консольного тестування проєкту за допомогою SonarLint:

Категорія	До рефакторингу	Після рефакторингу
Code Smells (критичні)	4	0
Security Hotspots	2	0
Bugs	1	0



```

PS C:\Users\Admin\VSCodeProjects\OPI_Labs> ruff check .
All checks passed!
PS C:\Users\Admin\VSCodeProjects\OPI_Labs>

```

Рис 3. Вікно SonarLint у VS Code

- **4.2. Регресійне тестування:** Запущено набір автоматизованих тестів за допомогою фреймворку `pytest`. Усі 20 існуючих тестів пройдено успішно (100% passed). Зміни в архітектурі не призвели до порушення існуючої бізнес-логіки.

```
PS C:\Users\Admin\VSCProjects\OPI_Labs> pytest
===== test session starts =====
platform win32 -- Python 3.10.6, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\Admin\VSCProjects\OPI_Labs
plugins: anyio-4.12.1
collected 20 items

tests\test_garden_service.py .....

===== 20 passed in 0.07s =====
PS C:\Users\Admin\VSCProjects\OPI_Labs>
```

Рис 4. Термінал з результатом pytest

- **4.3. Цикломатична складність:** За допомогою утиліти radon перевірено структурну складність модуля database.py. Отримана середня оцінка «А», що є еталонним результатом та підтверджує високу читабельність і відсутність надмірних розгалужень у коді.

```
PS C:\Users\Admin\VSCProjects\OPI_Labs> radon cc models.py -a -s
models.py
C 7:0 UserModel - A (2)
M 15:4 UserModel.is_credentials_present - A (2)
C 21:0 Contact - A (2)
C 36:0 VirtualPlant - A (2)
M 48:4 VirtualPlant.animate_growth - A (2)
M 54:4 VirtualPlant.wither - A (2)
M 10:4 UserModel.__init__ - A (1)
M 18:4 UserModel.to_dict - A (1)
M 24:4 Contact.__init__ - A (1)
M 29:4 Contact.to_dict - A (1)
M 41:4 VirtualPlant.__init__ - A (1)
M 59:4 VirtualPlant.to_dict - A (1)

12 blocks (classes, functions, methods) analyzed.
Average complexity: A (1.5)
```

```
PS C:\Users\Admin\VSCProjects\OPI_Labs> radon cc notifications.py -a -s
notifications.py
C 9:0 NotificationManager - A (2)
M 16:4 NotificationManager.is_rate_limited - A (2)
M 21:4 NotificationManager.send_push - A (2)
M 12:4 NotificationManager.__init__ - A (1)
M 33:4 NotificationManager.check_if_reminder_needed - A (1)

5 blocks (classes, functions, methods) analyzed.
Average complexity: A (1.6)
```

Рис 5-6. Термінал з результатами radon cc

5. Висновок

Під час виконання лабораторної роботи №4 було практично освоєно інструментарій колективної розробки — створено та проведено взаємний Code Review через механізм GitHub Pull Requests. Проаналізовано код напарника та сформульовано 5 структурованих зауважень щодо продуктивності, безпеки та керування ресурсами. Виконано рефакторинг власних модулів models.py та notifications.py, у результаті якого усунено порушення принципу єдиної

відповідальності (SRP), оптимізовано найменування об'єктів та очищено систему від невикористаного (мертвого) коду. Коректність внесених змін успішно верифіковано комплексом заходів: регресійні тести пройдено у повному обсязі (20/20), статичний аналізатор SonarLint підтверджує відсутність вразливостей, а оцінка цикломатичної складності (Radon) відповідає рівню «А». Рефакторинг дозволив підвищити стійкість та легкість підтримки програмного забезпечення.